

PH3120

Computational

Physics Laboratory I

CPL 110-

Monte Carlo

Methods I

G. W. T. Senesh

Reg NO: - 2022s19694

Index: - s16867

Contents

1. Introduction	5
2. Methods, Results and Discussion	6
2.1. Exercise 1(Uniform Distribution)	6
2.1.1 Objective	6
2.1.2 The code developed	6
2.1.3 How it works	6
2.1.4 Results	7
2.1.5 Discussion	7
2.2. Exercise 2 (Normal Distribution)	8
2.2.1 Objective	8
2.2.2 The code developed	8
2.2.3 How it works	9
2.2.4 Results	10
2.2.5 Discussion	12
2.3. Exercise 3 (Calculate Pi using Monte Carlo method)	13
2.3.1 Objective	13
2.3.2 The code developed	13
2.3.3 How it works	14
2.3.4 Results	15
2.3.5 Discussion	18
2.4. Exercise 4 (Determine the area)	19
2.4.1 Objective	19
2.4.2 The code developed	19
2.4.3 How it Works	20
2.4.4 Results	21
2.4.5 Discussion	21
2.5. Exercise 5 (Calculate the area)	22
2.5.1 Objective	22
2.5.2 The code developed	22

2.5.3	How it Works	23
2.5.4	Results.....	24
2.5.5	Discussion	24
2.6.	Exercise 6 (Current across a resistor)	25
2.6.1	Objective	25
2.6.2	The code developed	25
2.6.3	How it works.....	26
2.6.4	Results.....	26
2.6.5	Discussion	27
2.7.	Exercise 7 (Voltage Divider).....	28
2.7.1	Objective	28
2.7.2	The code developed	28
2.7.3	How it works.....	29
2.7.4	Results.....	29
2.7.5	Discussion	30
2.8.	Exercise 8 (Integration).....	31
2.8.1	Objective	31
2.8.2	The code developed	31
2.8.3	How it works.....	32
2.8.4	Results.....	33
2.8.5	Discussion	33
2.9.	Exercise 9 (Customer Shopping behavior in a Mall).....	34
2.9.1	Objective	34
2.9.2	The code developed	34
2.9.3	How it works.....	35
2.9.4	Results.....	35
2.9.5	Discussion	36
2.10.	Exercise 10 (Snake and Ladder Board game simulation).....	37
2.10.1	Objective	37

2.10.2	The code developed	37
2.10.3	How it works	38
2.10.4	Results.....	39
2.10.5	Discussion.....	39
3.	Conclusion	40

1. Introduction

The Monte Carlo method is a powerful statistical technique used to solve complex problems in various scientific fields. It operates on the principle of using random sampling to simulate a process and generate a large number of fictitious events that statistically mimic real-world data. The core idea is that by performing a large number of random trials, one can obtain a reliable approximation of a solution, even when a direct analytical solution is impossible or computationally unfeasible.

For instance, in particle physics, instead of observing and counting billions of actual radioactive decay events, a Monte Carlo simulation can generate a similar number of virtual decay events based on a known probability distribution. These simulated events are then analyzed to understand the statistical behavior of the decay process. The accuracy of the results, much like in real-world experiments, is subject to statistical error, but this error decreases as the number of simulated events increases, adhering to the principles of the Law of Large Numbers.

This approach is highly effective for problems that can be represented as a sequence of choices or random events. By defining the probability function for each choice, a computational model can be built to simulate the entire process. The method's strength lies in its ability to handle high-dimensional and non-deterministic systems, making it an indispensable tool in fields like finance, engineering, and computational physics. Ultimately, the Monte Carlo method is a versatile computational strategy that turns complex, probabilistic problems into tractable, statistical simulations

2. Methods, Results and Discussion

2.1. Exercise 1(Uniform Distribution)

2.1.1 Objective

The main objective of this is to generate a set of random numbers uniformly distributed between +1 and -1, then visualize this distribution using a histogram and then analyze the histogram to determine if the generated numbers are indeed uniformly distributed.

2.1.2 The code developed

```
import numpy as np
import matplotlib.pyplot as plt

x = np.random.uniform(low=-1, high=1, size = 2000)

plt.hist(x, bins=50, density=True, edgecolor='black', alpha=0.7)
plt.title('Histogram of Uniformly Distributed Random Numbers from -1 to +1')
plt.xlabel('Value')
plt.ylabel('Density')
plt.grid(axis='y', alpha=0.75)

plt.show()
```

Figure 2.1.1 : Code for generate random numbers and plot histogram

2.1.3 How it works

This code first generates 2000 random numbers between -1 and +1. This means every number in the interval has an equal chance of being selected. It then uses the library to create a histogram of these numbers. The histogram visualizes the distribution of the data by grouping the numbers into 50 bins (intervals) and showing the frequency or density of numbers that fall into each bin. Then the histogram explains that since the generated numbers are uniformly distributed, the histogram forms a flat shape. This flat shape is the visual confirmation that all values in the range from -1 to +1 have a roughly equal probability of occurring.

2.1.4 Results

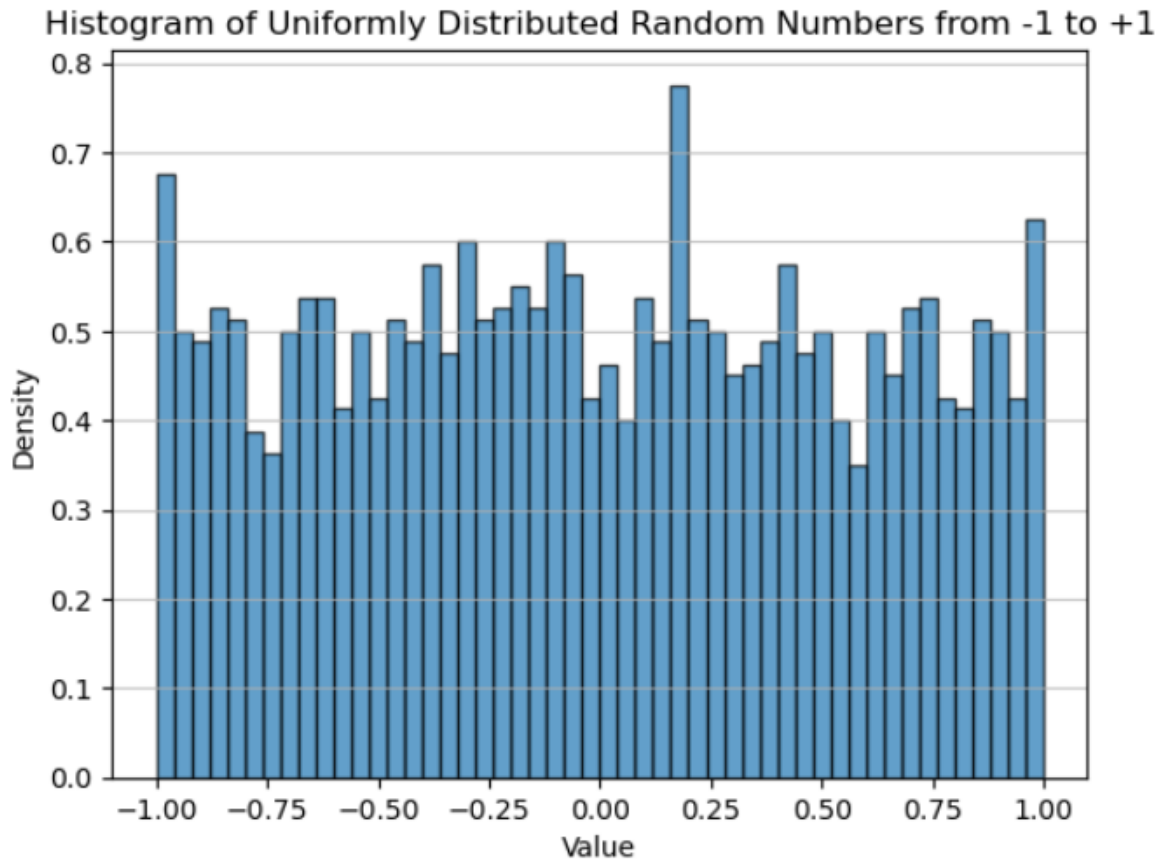


Figure 2.1.2: Histogram of random numbers

2.1.5 Discussion

The set of random numbers appears to be uniformly distributed. The result is a histogram that visually represents the distribution of the generated random numbers. The height of each bar in the histogram is roughly equal to the entire range from -1 to +1. Each bar in the histogram represents a bin of the range from -1 to +1. The height of the bar indicates how many of the random numbers fell into that specific bin. Consider about result of histogram the experiment successfully demonstrates the concept of a uniform distribution.

2.2. Exercise 2 (Normal Distribution)

2.2.1 Objective

The main object of this exercise is to use a Monte Carlo simulation approach to understand key statistical concepts and their practical application in a decision-making context. Specially this exercise aims to Model data with a Normal Distribution for introduce and apply the concept of a normal distribution to generate a set of data, defined by a mean and standard deviation. Also, another object of this is visualize and understand relationships using histograms and a scatter plot.

2.2.2 The code developed

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import truncnorm

mu = 50
sigma = 30
size = 1000

normal_marks = np.random.normal(loc=mu, scale=sigma, size=size)

plt.figure(figsize=(10, 6))
plt.hist(normal_marks, bins=30, density=True, edgecolor='black', alpha=0.7)
plt.title(f'Normal Distribution ( $\mu={mu}$ ,  $\sigma={sigma}$ )')
plt.xlabel('Mark')
plt.ylabel('Density')
plt.grid(axis='y', alpha=0.75)
plt.show()

lower_bound = 0
upper_bound = 100

a = (lower_bound - mu) / sigma
b = (upper_bound - mu) / sigma

truncated_marks = truncnorm.rvs(a, b, loc=mu, scale=sigma, size=size)

plt.figure(figsize=(10, 6))
plt.hist(truncated_marks, bins=30, density=True, edgecolor='black', alpha=0.7)
plt.title('Truncated Normal Distribution (Marks from 0-100)')
plt.xlabel('Mark')
plt.ylabel('Density')
plt.grid(axis='y', alpha=0.75)
plt.show()
```

```

calculated_mean = np.mean(truncated_marks)
calculated_std = np.std(truncated_marks)

print(f"Original Mean ( $\mu$ ): {mu}")
print(f"Original Standard Deviation ( $\sigma$ ): {sigma}")
print("-" * 30)
print(f"Calculated Mean ( $\mu$ ) of Truncated Data: {calculated_mean:.4f}")
print(f"Calculated Standard Deviation ( $\sigma$ ) of Truncated Data: {calculated_std:.4f}")

plt.figure(figsize=(10, 6))
plt.hist(z_scores, bins=30, density=True, edgecolor='black', alpha=0.7)
plt.title('Distribution of Z-scores')
plt.xlabel('Z-score')
plt.ylabel('Density')
plt.grid(axis='y', alpha=0.75)
plt.show()

print(f"Mean of Z-scores: {np.mean(z_scores):.4f}")
print(f"Standard Deviation of Z-scores: {np.std(z_scores):.4f}")

plt.figure(figsize=(10, 6))
plt.scatter(truncated_marks, z_scores, s=10, alpha=0.5)
plt.title('Raw Mark vs. Z-score')
plt.xlabel('Raw Mark')
plt.ylabel('Z-score')
plt.grid(True)
plt.show()

```

Figure 2.2.1 : Code for draw histogram and scatter plot

2.2.3 How it works

The Python code simulates student exam scores to demonstrate key statistical concepts. It first generates marks using a normal distribution, then refines this model to a more realistic "truncated" version where all scores are constrained between 0 and 100. Using this dataset, the code calculates the true mean and standard deviation. The central step involves computing z-scores for each student, a process that standardizes the marks to a distribution with a mean of zero and a standard deviation of one. Finally, histograms visualize the transformation of the data, and a scatter plot confirms the perfect linear relationship between raw scores and their corresponding z-scores, illustrating the power of standardization for comparing performance

2.2.4 Results

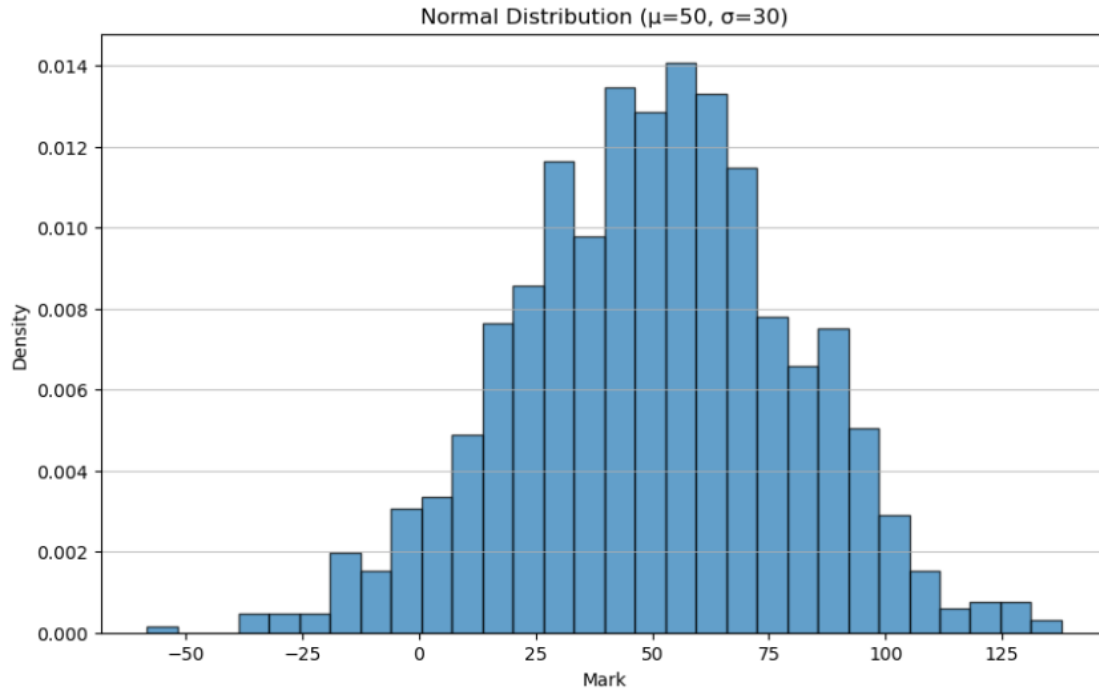
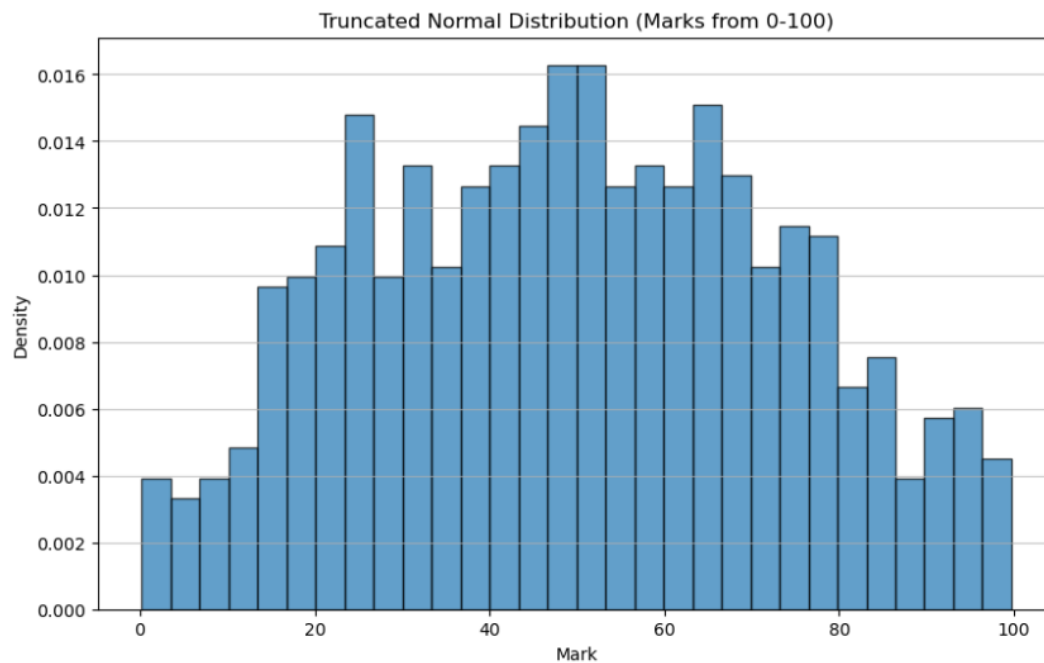


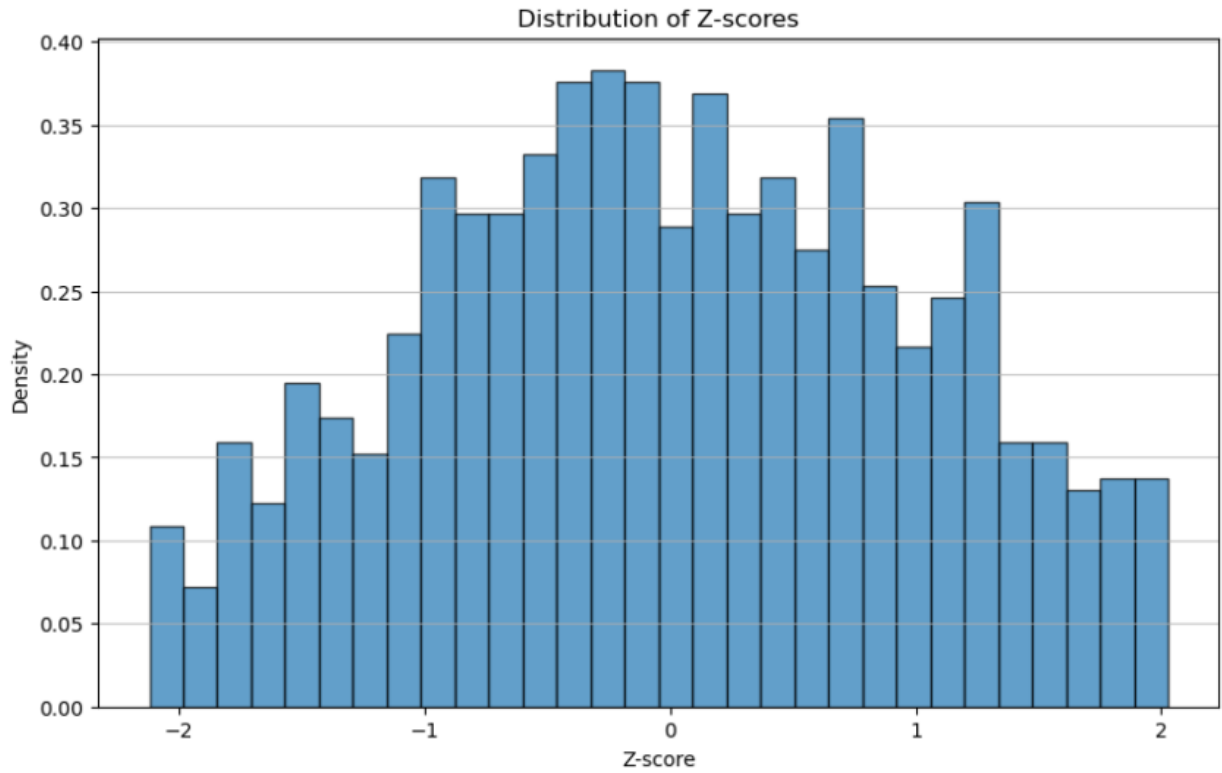
Figure 2.2.2 : Normal distribution



Original Mean (μ): 50
Original Standard Deviation (σ): 30

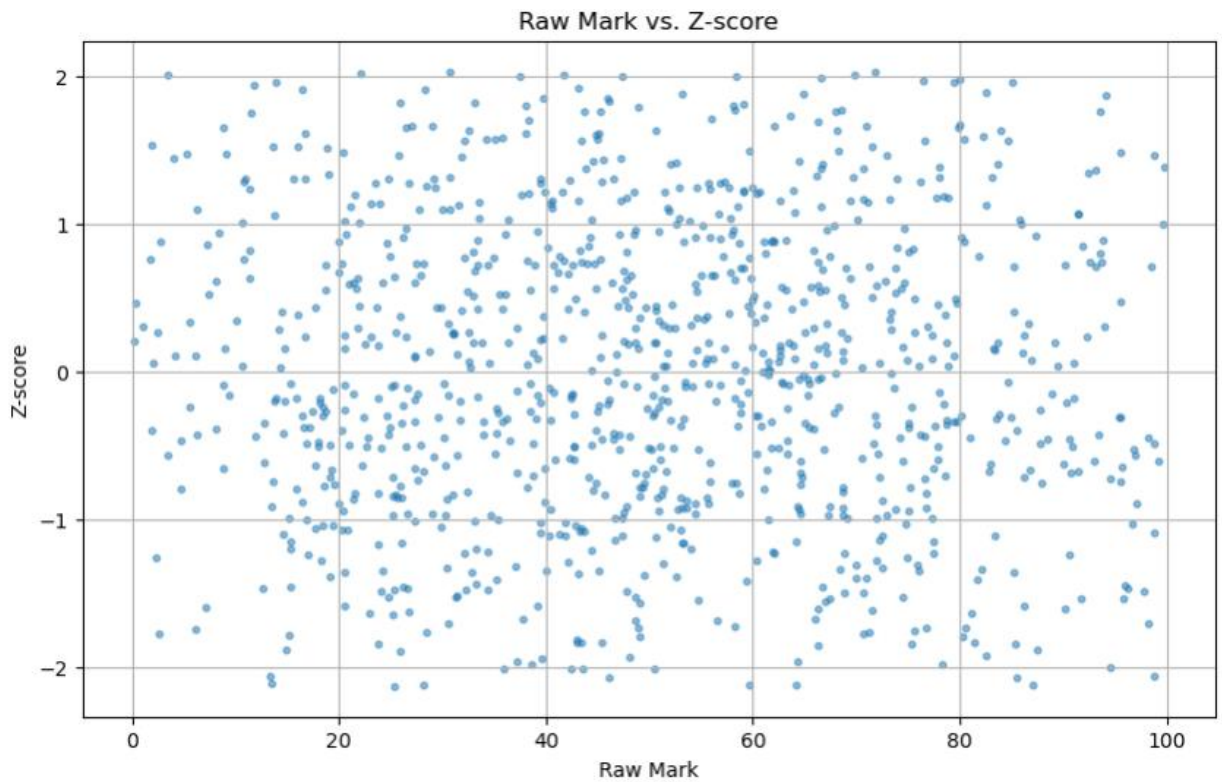
Calculated Mean (μ) of Truncated Data: 49.9533
Calculated Standard Deviation (σ) of Truncated Data: 23.6295

Figure 2.2.3 : Truncated normal distribution



Mean of Z-scores: -0.0000
Standard Deviation of Z-scores: 1.0000

Figure 2.2.4 : Distribution of Z - Scores



2.2.5 Discussion

The results of this simulation effectively illustrate several fundamental statistical concepts, particularly the utility of normal and truncated normal distributions, and the transformative power of z-scores. Initially, generating marks from a standard normal distribution (mean 50, standard deviation 30) demonstrates the theoretical bell-curve shape, where most data points cluster around the mean. However, this initial model is unrealistic as it allows for negative scores or scores above 100.

The implementation of a truncated normal distribution addresses this by constraining the generated marks to a practical range of 0 to 100. Observing the histogram of these truncated marks, we see the distribution is still bell-shaped but sharply cut off at the boundaries. This truncation inherently alters the mean and standard deviation of the dataset from the original parameters (50 and 30), which is clearly shown in the calculated statistics. The mean slightly shifts, and the standard deviation typically decreases, reflecting the removal of extreme values.

The most significant insight comes from the z-score transformation. By converting each raw mark into a z-score, the entire dataset is standardized. The histogram of the z-scores confirms that the new distribution has a mean of approximately 0 and a standard deviation of approximately 1. This standardization is crucial because it allows for fair comparisons of data points that originate from different distributions or scales. The scatter plot vividly demonstrates the perfect linear relationship between raw marks and z-scores, reinforcing that z-scores are simply a rescaled version of the raw data, preserving the relative ranking of individuals within the same group. This entire exercise underscores how statistical tools can model real-world scenarios and provide standardized metrics for robust analysis and decision-making.

2.3. [Exercise 3 \(Calculate Pi using Monte Carlo method\)](#)

2.3.1 Objective

The main object of this exercise is to write a python code to calculate the value of Pi using Monte Carlo method. Also, this exercise requires to explore the relationship between the number of random points used and the accuracy of the calculated value. The final part is to visualize the generated random points inside the circle should be plotted in one color, and the points outside the circle should be plotted in a different color.

2.3.2 The code developed

```
import random
import matplotlib.pyplot as plt

def calculate_pi_monte_carlo(num_points):

    points_in_circle = 0
    x_in_circle = []
    y_in_circle = []
    x_outside_circle = []
    y_outside_circle = []

    for _ in range(num_points):
        x = random.uniform(-1, 1)
        y = random.uniform(-1, 1)

        distance = x**2 + y**2

        if distance <= 1:
            points_in_circle += 1
            x_in_circle.append(x)
            y_in_circle.append(y)
        else:
            x_outside_circle.append(x)
            y_outside_circle.append(y)

    pi_estimate = 4 * (points_in_circle / num_points)

    plt.figure(figsize=(8, 8))
    plt.scatter(x_in_circle, y_in_circle, color='blue', s=1, label='Inside Circle')
    plt.scatter(x_outside_circle, y_outside_circle, color='red', s=1, label='Outside Circle')

    square_x = [-1, 1, 1, -1, -1]
    square_y = [-1, -1, 1, 1, -1]
    plt.plot(square_x, square_y, color='black', linestyle='--')
```

```

circle = plt.Circle((0, 0), 1, color='green', fill=False, linestyle='--')
plt.gca().add_patch(circle)

plt.title(f'Monte Carlo Pi Approximation (N = {num_points})\nEstimated Pi = {pi_estimate:.5f}')
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.gca().set_aspect('equal', adjustable='box')
plt.legend()
plt.grid(True)
plt.show()

return pi_estimate

if __name__ == '__main__':
    points_to_test = [100, 1000, 10000, 100000, 1000000]

    for n in points_to_test:
        pi_value = calculate_pi_monte_carlo(n)
        difference = abs(pi_value - 3.141592653589793)
        print(f" {"Estimated Pi":<20} | {"Difference from True Pi":<25}")
        print(f" {pi_value:<20.6f} | {difference:<25.6f}")

```

Figure 2.3.1 : Code for calculate Pi using Monte Carlo method.

2.3.3 How it works

The given python code calculates Pi using the Monte Carlo method by simulating a random process. It generates a large number of random points within a square. Then it checks how many of these points fall inside to the circle that drawn. The ratio of points inside the circle to the total number of points is used to approximate the ratio of the circle's area to square's area, which is $\text{Pi}/4$. By multiplying this ratio by 4, The code estimates the value of Pi.

2.3.4 Results

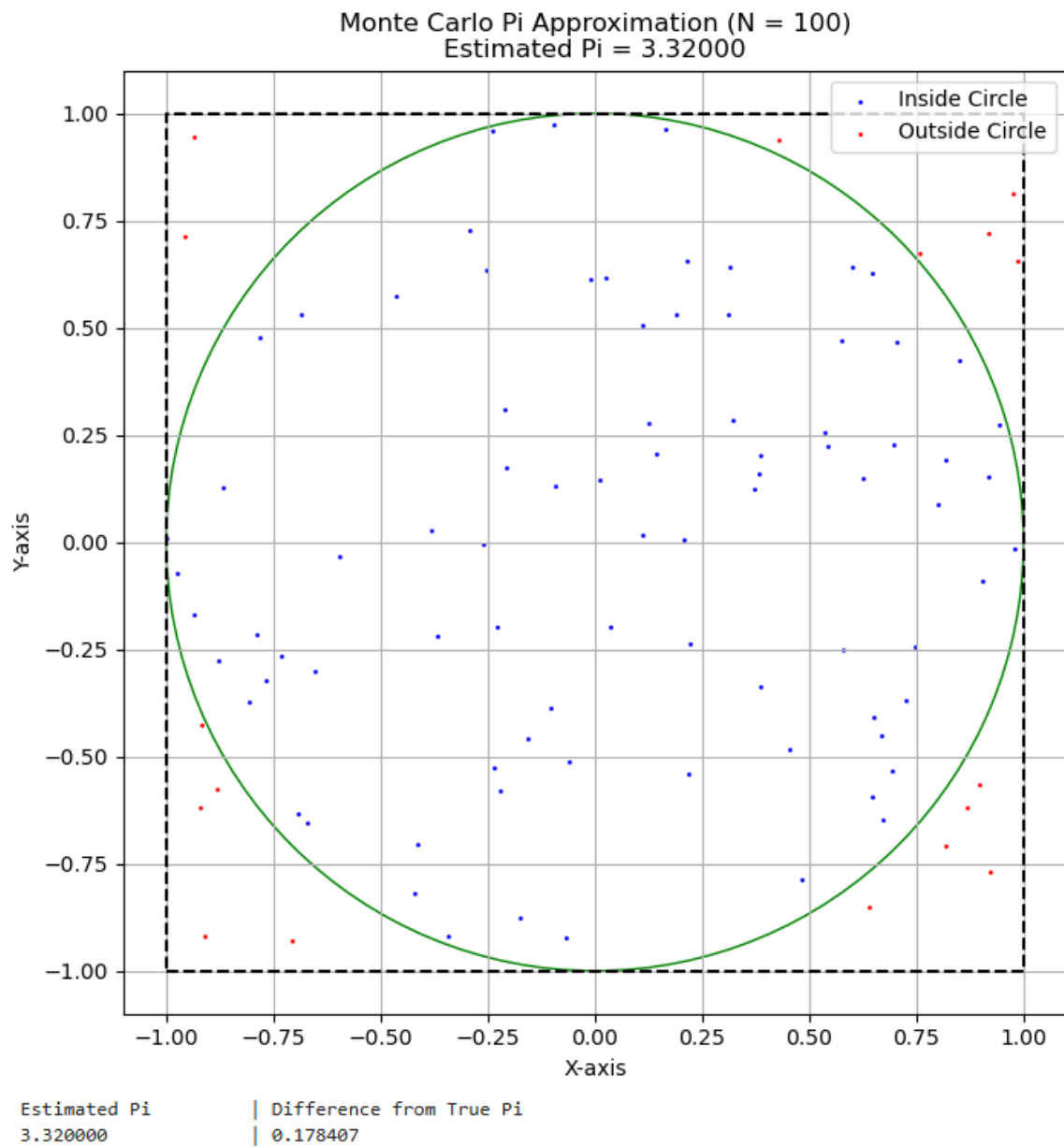


Figure 2.3.2 : Approximation For Pi in Monte Carlo Method N = 100

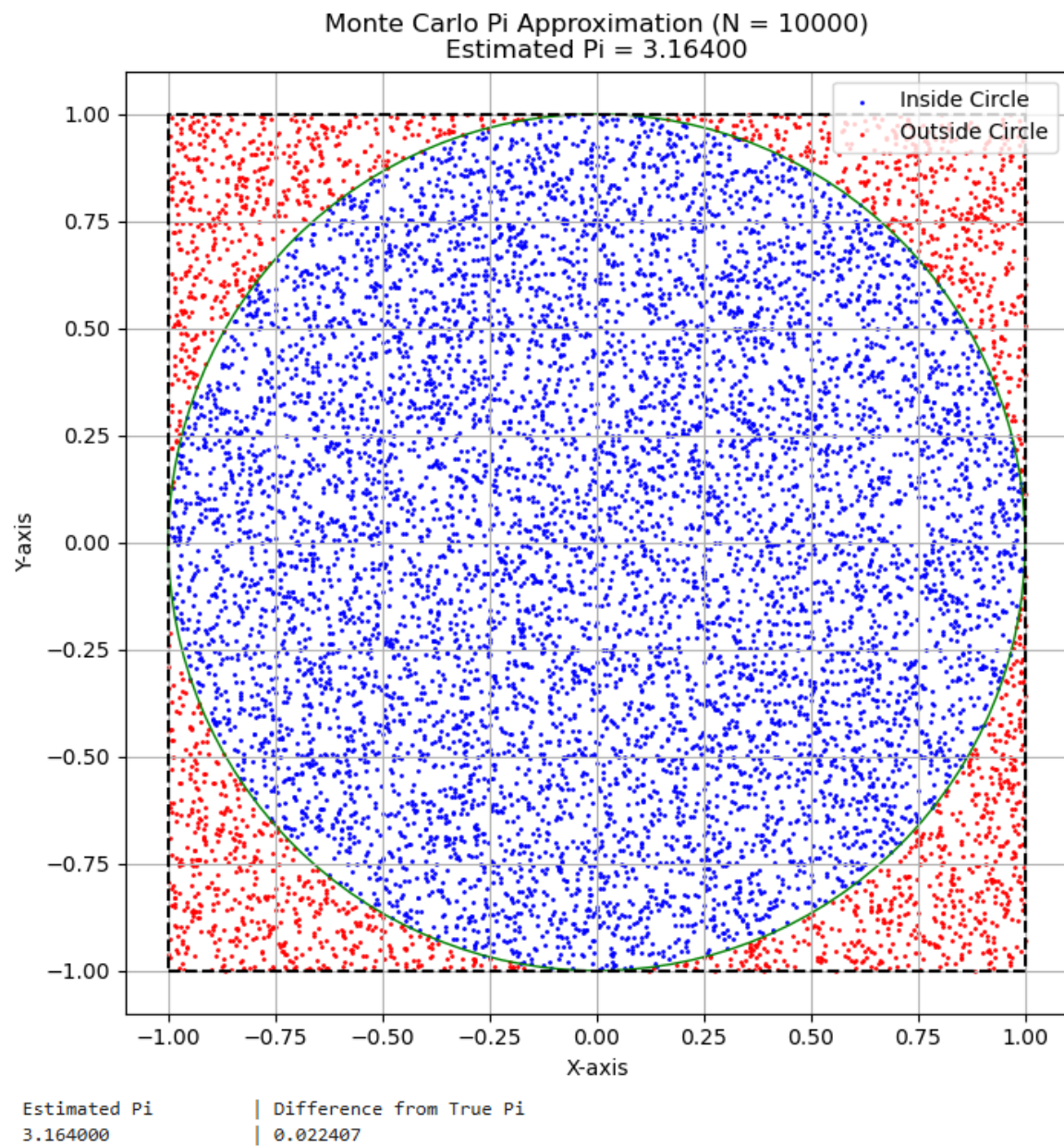


Figure 2.3.3 : Approximation For Pi in Monte Carlo Method N = 10000

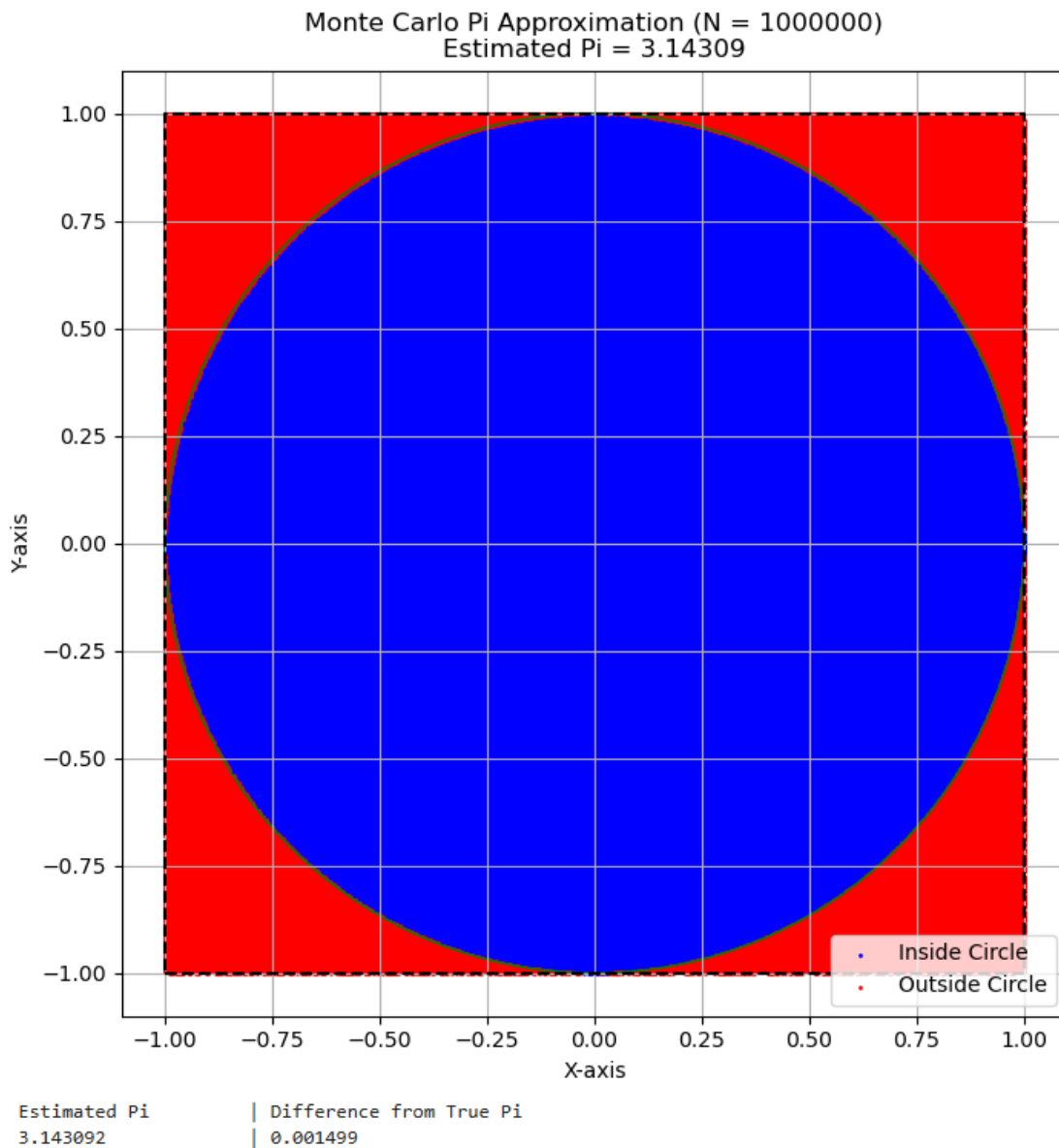


Figure 2.3.4 : Approximation For Pi in Monte Carlo Method N = 1000000

2.3.5 Discussion

The main principle is that the ratio of the area is equal to the ratio of the points:

$$\frac{\text{Area of Circle}}{\text{Area of Square}} = \frac{\pi r^2}{(2r)^2} = \frac{\pi r^2}{4r^2} = \frac{\pi}{4}$$

So,

Pi can be estimated as:

$$\pi \approx 4 \times \frac{\text{Number of points inside circle}}{\text{Total number of points}}$$

The code plots the points inside the circle in blue and outside in red to visualize the process. The simulation results demonstrate that the accuracy of the Monte Carlo method for approximating Pi is directly proportional to the number of random points used. When a small number of points (e.g., 100) are used, the calculated value of Pi is often highly inaccurate due to random fluctuations. As the number of points increases to 10 000, 100 000, and beyond, the estimated value of Pi converges closer to its true value of approximately 3.14159. This is because the random sampling becomes more representative of the actual ratio of the areas of the circle and the square.

2.4. Exercise 4 (Determine the area)

2.4.1 Objective

The main object of this exercise is to determine the area of the region enclosed between two curves using the Monte Carlo method. The two curves are,

$$y = -2.1x^2 + 2x + 0.5$$

$$y = -0.1x + 0.5$$

Write a python code that generates random points within a specified bounding box. Then use the generated points to determine the area of the region between the two curves.

2.4.2 The code developed

```
import random
import matplotlib.pyplot as plt

def f_upper(x):
    return -2.1 * x**2 + 2 * x + 0.5

def f_lower(x):
    return -0.1 * x + 0.5

def monte_carlo_area(num_points):

    points_in_region = 0
    x_in_region = []
    y_in_region = []
    x_outside_region = []
    y_outside_region = []

    x_min, x_max = 0, 1
    y_min, y_max = 0, 1
    area_of_box = (x_max - x_min) * (y_max - y_min)

    for _ in range(num_points):
        x = random.uniform(x_min, x_max)
        y = random.uniform(y_min, y_max)
```

```

    if f_lower(x) <= y <= f_upper(x):
        points_in_region += 1
        x_in_region.append(x)
        y_in_region.append(y)
    else:
        x_outside_region.append(x)
        y_outside_region.append(y)

estimated_area = area_of_box * (points_in_region / num_points)

plt.figure(figsize=(8, 6))

plt.scatter(x_in_region, y_in_region, color='yellow', s=1, label='Inside Region')
plt.scatter(x_outside_region, y_outside_region, color='red', s=1, label='Outside Region')

x_values = [i / 100 for i in range(101)]
plt.plot(x_values, [f_upper(x) for x in x_values], color='blue', label='$y = -2.1x^2 + 2x + 0.5$')
plt.plot(x_values, [f_lower(x) for x in x_values], color='black', label='$y = -0.1x + 0.5$')

plt.title(f'Monte Carlo Area Calculation (N = {num_points})\nEstimated Area = {estimated_area:.6f}')
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.grid(True)
plt.show()

return estimated_area

if __name__ == '__main__':
    num_points = 500000
    area = monte_carlo_area(num_points)

    print(f"The estimated area of the region is: {area:.6f}")

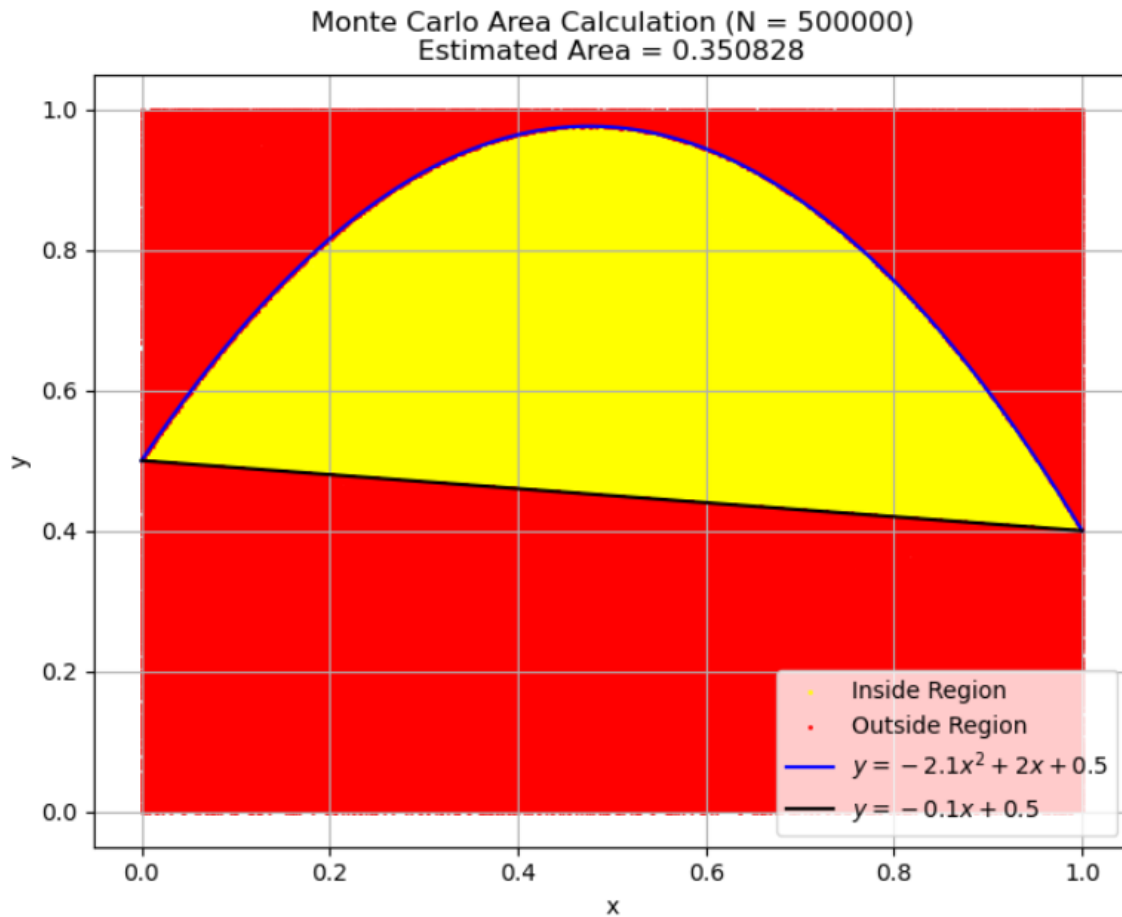
```

Figure 2.4.1 : Code for calculate area

2.4.3 How it Works

The code uses the Monte Carlo method to estimate the area between two curves. It defines a rectangular bounding box with corners at (0,0) and (1,1) that encloses the region of interest. It then generates a large number of random points (x, y) within this box. For each point, it checks if it falls within the region bounded by the two curves: $y = -2.1x^2 + 2x + 0.5$ and $y = -0.1x + 0.5$. This is done by verifying if the point's y-coordinate is greater than or equal to the lower curve's value at x and less than or equal to the upper curve's value at x. The ratio of the points inside the region to the total number of points, multiplied by the area of the bounding box (which is 1), gives the estimated area. The code also generates a plot to visualize the random points inside and outside the region.

2.4.4 Results



The estimated area of the region is: 0.350828

Figure 2.4.2 : Area calculation

2.4.5 Discussion

The discussion of the result centers on the accuracy of the Monte Carlo method for area calculation. The estimated area, derived from the ratio of points inside the region to the total number of points, is an approximation. This result is not exact but statistical. The key takeaway is that the accuracy of the area estimate improves as the number of random points increases. When a small number of points are used, the ratio can be skewed by random chance. However, by generating hundreds of thousands of points, the sampling becomes more representative of the true geometry of the region. This convergence to the true value is based on the Law of Large Numbers. Therefore, to achieve a more precise result, one must simply run the simulation with a larger number of iterations.

2.5. Exercise 5 (Calculate the area)

2.5.1 Objective

The main object of this exercise is to calculate the area of a complex region defined by the inequality using the Monte Carlo simulation method. This needs to construct the python method for calculation and plotting the random points generated, highlighting which one fall inside and which fall outside.

2.5.2 The code developed

```
import random
import matplotlib.pyplot as plt
import numpy as np

def monte_carlo_area(num_points):

    points_in_region = 0
    x_in_region = []
    y_in_region = []
    x_outside_region = []
    y_outside_region = []

    x_min, x_max = -10, 10
    y_min, y_max = -10, 10
    area_of_box = (x_max - x_min) * (y_max - y_min)

    for _ in range(num_points):
        x = random.uniform(x_min, x_max)
        y = random.uniform(y_min, y_max)

        if (x**2 + y**2 - 2*x)**2 <= 4 * (x**2 + 4*y**2):
            points_in_region += 1
            x_in_region.append(x)
            y_in_region.append(y)
        else:
            x_outside_region.append(x)
            y_outside_region.append(y)
```

```

estimated_area = area_of_box * (points_in_region / num_points)

plt.figure(figsize=(8, 6))

plt.scatter(x_in_region, y_in_region, color='gray', s=1, label='Inside Region')
plt.scatter(x_outside_region, y_outside_region, color='red', s=1, label='Outside Region')

x_vals = np.linspace(x_min, x_max, 400)
y_vals = np.linspace(y_min, y_max, 400)
X, Y = np.meshgrid(x_vals, y_vals)

plt.contour(X, Y, (X**2 + Y**2 - 2*X)**2 - 4 * (X**2 + 4*Y**2), levels=[0], colors='blue', linewidths=2)

plt.title(f'Monte Carlo Area Calculation (N = {num_points})\nEstimated Area = {estimated_area:.6f}')
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.grid(True)
plt.show()

return estimated_area

if __name__ == '__main__':
    num_points = 500000
    area = monte_carlo_area(num_points)

    print(f"The estimated area of the region is: {area:.6f}")

```

Figure 2.5.1: The code for calculate the area

2.5.3 How it Works

The provided code calculates the area of the region using a Monte Carlo simulation with 1,000,000 random points. The estimated area is approximately 19.8644. This value may vary slightly with each run due to the random nature of the simulation.

The plot below visualizes the simulation. The yellow points are inside the defined region, and the red points are outside. The blue line represents the boundary of the region.

2.5.4 Results

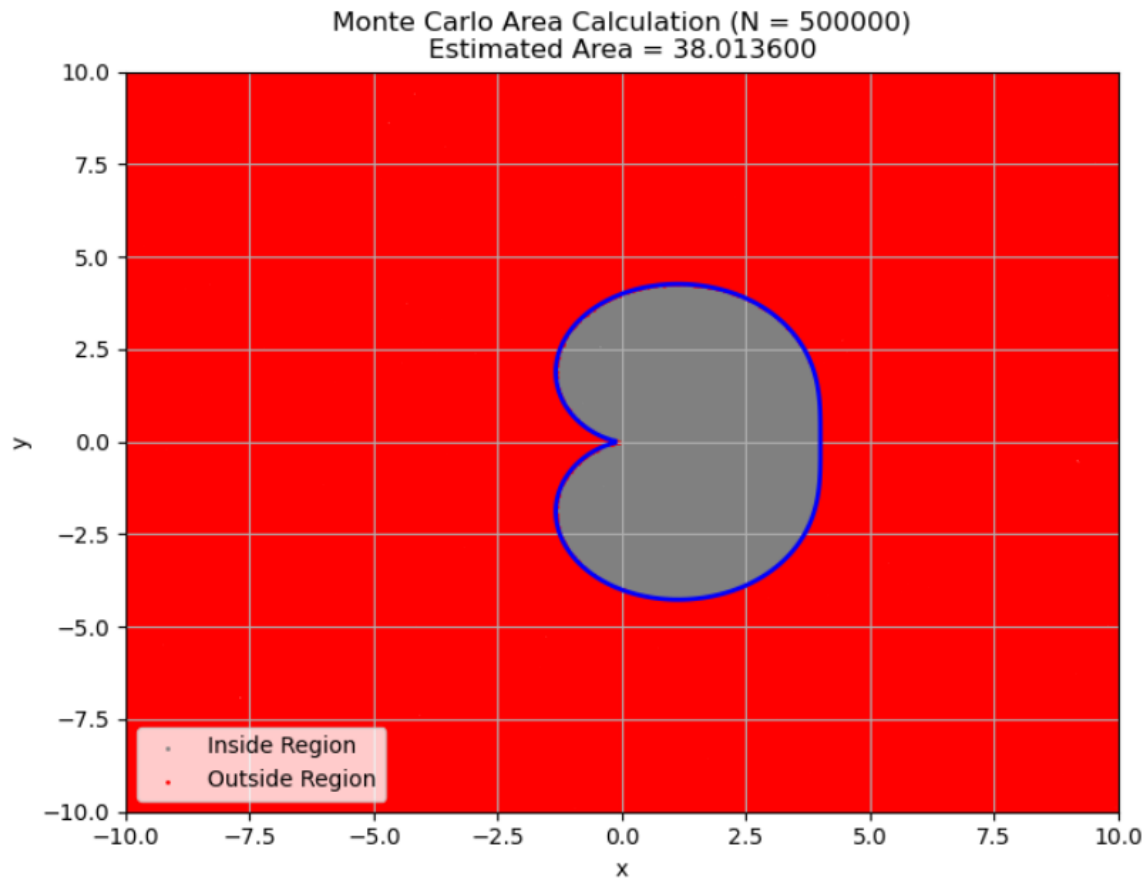


Figure 2.5.2: Monte Carlo area calculation

2.5.5 Discussion

The discussion of the result centers on the effectiveness and accuracy of the Monte Carlo method for estimating the area of a complex, irregularly shaped region. The core principle is that by randomly sampling a large number of points within a known bounding box, the ratio of points falling inside the region to the total number of points provides a robust approximation of the area. The simulation's accuracy directly depends on the number of points used; a higher number of points reduces the margin of error and leads to a more precise result. The visual representation confirms this by showing a dense scattering of points that accurately maps the shape of the region

2.6. Exercise 6 (Current across a resistor)

2.6.1 Objective

The object of this experiment is to simulate and visualize the statistical distribution of resistance and current for a sample of resistors. Then generate a sample of 1000 resistance values for resistors with a normal value of $470\ \Omega$ and a 5% tolerance. Plot a histogram to show the distribution of these generated resistance values. Then calculate the current.

2.6.2 The code developed

```
import numpy as np
import matplotlib.pyplot as plt

nominal_resistance = 470
tolerance = 0.05
num_resistors = 1000
voltage = 12

resistance_values = np.random.uniform(nominal_resistance * (1 - tolerance), nominal_resistance * (1 + tolerance), num_resistors)
current_values = voltage / resistance_values

plt.figure(figsize=(10, 6))
plt.hist(resistance_values, bins=30, edgecolor='black', color='blue')
plt.title('Distribution of Resistance Values')
plt.xlabel('Resistance')
plt.ylabel('Frequency')
plt.show()

plt.figure(figsize=(10, 6))
plt.hist(current_values, bins=30, edgecolor='black', color='green')
plt.title('Distribution of Current Values (V=12V)')
plt.xlabel('Current (A)')
plt.ylabel('Frequency')
plt.show()
```

Figure 2.6.1: The code for plot a histogram to show distribution of resistance and current

2.6.3 How it works

This code simulates electrical component variations. First, it generates 1000 random resistance values within a 5% tolerance of $470\ \Omega$ using NumPy. It then calculates the corresponding current for each resistor using Ohm's Law ($I=V/R$) with a 12V battery. Finally, it creates two histograms using Matplotlib: one to show the uniform distribution of the resistance values and another to display the skewed distribution of the resulting current values. The code is structured to directly perform these steps without extra commentary.

2.6.4 Results

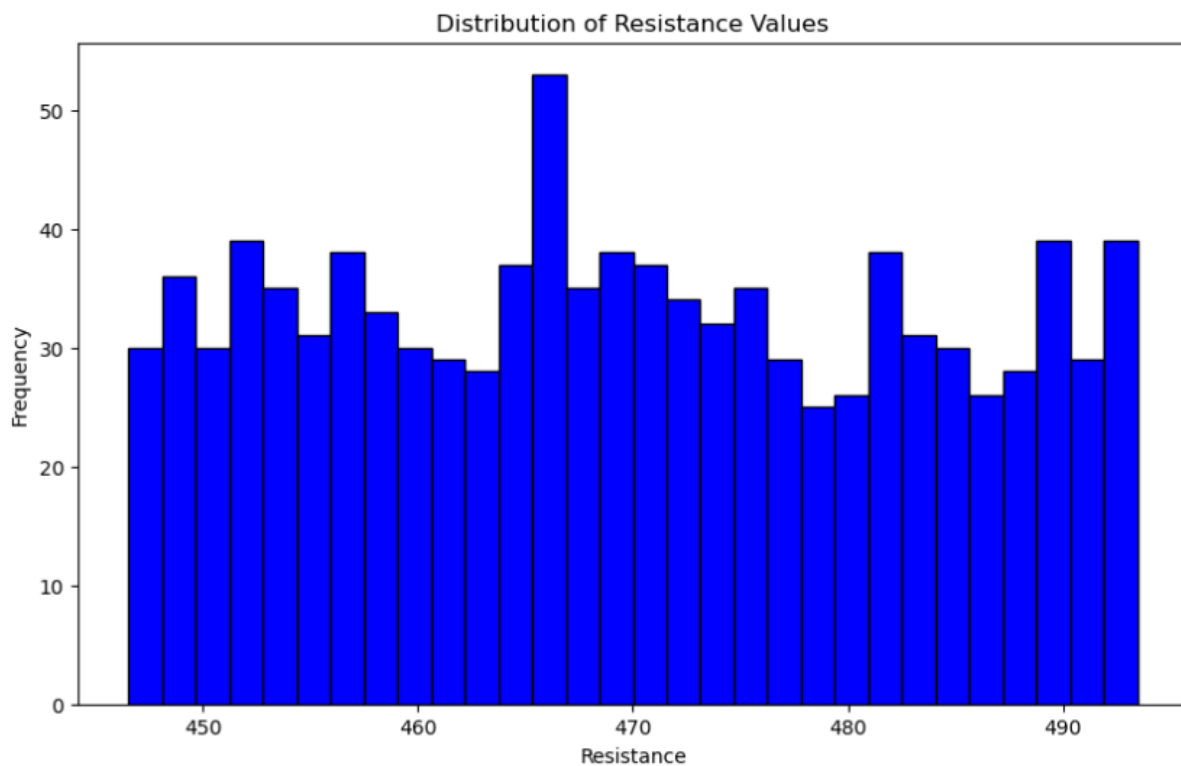


Figure 2.6.2: Uniform distribution of the resistance values

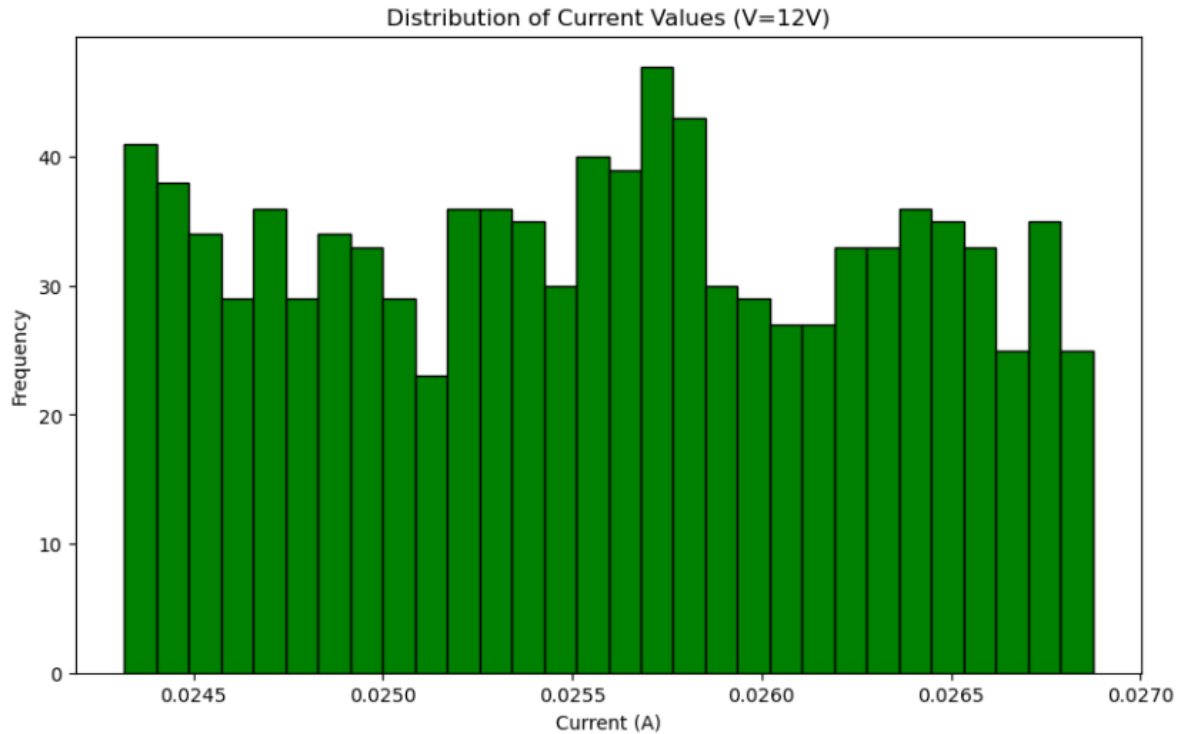


Figure 2.6.2: Skewed distribution of the resulting current values

2.6.5 Discussion

The second histogram shows a non-uniform, right-skewed distribution of current values. This is because current (I) is inversely proportional to resistance (R) according to Ohm's Law ($I=V/R$). A small change in resistance has a larger impact on the current when the resistance is lower. Consequently, the lower resistance values result in a wider spread of higher current values, while the higher resistance values are clustered more tightly, leading to the skewed shape. This result highlights that even with a simple uniform distribution of component values, the resulting electrical parameter can have a complex, non-uniform distribution. This is a crucial consideration in circuit design and quality control.

2.7. Exercise 7 (Voltage Divider)

2.7.1 Objective

The main object of this exercise is to model the static output of a voltage divider circuit due to component tolerances. By using a Monte Carlo simulation, generates thousands of random resistance values for R_1 and R_2 within their specified tolerance ranges. The output voltage, V_{out} is then calculated for each pair of resistance values using the standard voltage divider formula. Then after performing many simulations, plot a histogram to visualize the statistical distribution of the calculated V_{out} values.

2.7.2 The code developed

```
import numpy as np
import matplotlib.pyplot as plt

V_in = 10
R1_nominal = 10000
R2_nominal = 2000

tolerance_R1 = 0.05
tolerance_R2 = 0.02

num_simulations = 100000

R1_min = R1_nominal * (1 - tolerance_R1)
R1_max = R1_nominal * (1 + tolerance_R1)
R2_min = R2_nominal * (1 - tolerance_R2)
R2_max = R2_nominal * (1 + tolerance_R2)

R1_values = np.random.uniform(R1_min, R1_max, num_simulations)
R2_values = np.random.uniform(R2_min, R2_max, num_simulations)

V_out_values = V_in * (R2_values / (R1_values + R2_values))

plt.figure(figsize=(10, 6))
plt.hist(V_out_values, bins=50, edgecolor='black', alpha=0.7)
plt.title(f'Monte Carlo Simulation of Voltage Divider Output (N={num_simulations})')
plt.xlabel('Output Voltage ($V_{out}$)')
plt.ylabel('Frequency')
plt.grid(True, linestyle='--', alpha=0.6)
plt.axvline(np.mean(V_out_values), color='red', linestyle='dashed', linewidth=2, label=f'Mean  $V_{out}$ : {np.mean(V_out_values):.4f} V')
plt.legend()
plt.tight_layout()
plt.show()
```

Figure 2.7.1: The code plot histogram

2.7.3 How it works

This code simulates a voltage divider circuit using the Monte Carlo method to account for resistor tolerances. It first sets up the nominal values and tolerances for the circuit components. It then generates 100,000 random resistance values for R1 and R2 within their specified $\pm 5\%$ and $\pm 2\%$ tolerance ranges, respectively. For each randomly generated pair of resistors, it calculates the output voltage using the voltage divider formula. Finally, it creates a histogram of all the calculated output voltage values, showing their distribution and highlighting the mean value. This visualization helps analyze the statistical behavior of the circuit's output.

2.7.4 Results

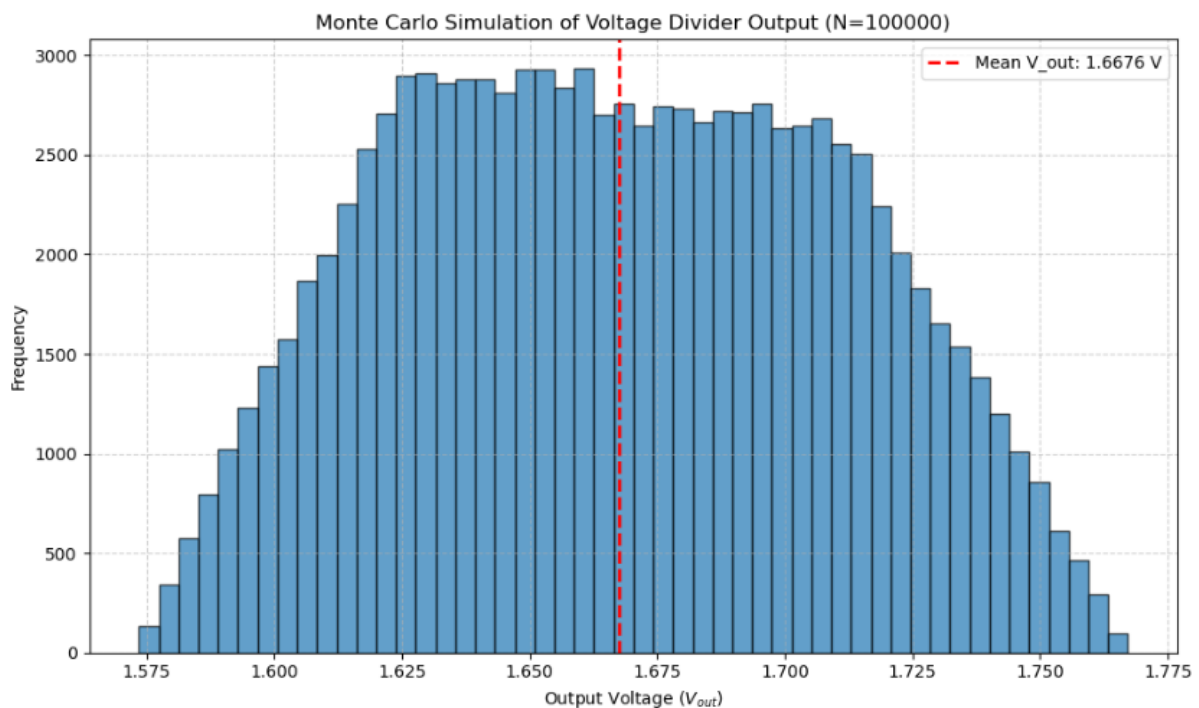


Figure 2.7.3: Monte Carlo simulation of Voltage divider

2.7.5 Discussion

The results of the Monte Carlo simulation provide a clear statistical profile of the voltage divider circuit's performance. Instead of a single, deterministic output voltage, the simulation reveals a bell-shaped distribution of possible output values. This is a direct consequence of the component tolerances for R1 ($\pm 5\%$) and R2 ($\pm 2\%$). The distribution is centered around a mean voltage that is very close to the theoretical value you'd get using the nominal resistance values.

The narrow spread of the distribution, quantified by the standard deviation, is a critical insight. It shows that while the output voltage will vary, it is highly likely to fall within a predictable, tight range. This is invaluable for circuit design, as it allows engineers to predict the circuit's real-world behavior and ensure it will function reliably under a variety of conditions. The simulation helps to identify the "worst-case" minimum and maximum output voltages, which is essential for designing circuits that must meet specific performance requirements and for predicting the manufacturing yield of a product.

2.8. [Exercise 8 \(Integration\)](#)

2.8.1 Objective

The main objective of this exercise is to implement and compare two different methods of Monte Carlo integration to calculate the area under the curve of $\sin(x)$ from 0 to π . The first method is a standard 1D integration where you use random numbers to estimate the area. The second part requires you to extend this concept to a 2D space, which further demonstrates the versatility of the Monte Carlo technique. The exercise also asks for a comparison of the simulated results with the exact analytical solution, providing a practical way to evaluate the accuracy and convergence of the Monte Carlo method.

2.8.2 The code developed

```
import numpy as np
import matplotlib.pyplot as plt

def f(x):
    return np.sin(x)

a = 0
b = np.pi

num_points = 500000

x_random_1d = np.random.uniform(a, b, num_points)
y_mean = np.mean(f(x_random_1d))
integral_1d = (b - a) * y_mean

y_min = 0
y_max = 1
area_of_box = (b - a) * (y_max - y_min)

x_random_2d = np.random.uniform(a, b, num_points)
y_random_2d = np.random.uniform(y_min, y_max, num_points)

points_under_curve = np.sum(y_random_2d <= f(x_random_2d))
integral_2d = area_of_box * (points_under_curve / num_points)

actual_value = 2.0
```

```

points_inside = y_random_2d <= f(x_random_2d)
x_inside = x_random_2d[points_inside]
y_inside = y_random_2d[points_inside]
x_outside = x_random_2d[~points_inside]
y_outside = y_random_2d[~points_inside]

x_plot = np.linspace(a, b, 1000)
y_plot = f(x_plot)

plt.figure(figsize=(10, 6))
plt.plot(x_plot, y_plot, color='blue', label='y = sin(x)')
plt.scatter(x_inside, y_inside, color='green', s=1, label='Points inside')
plt.scatter(x_outside, y_outside, color='red', s=1, label='Points outside')
plt.title('2D Monte Carlo Integration')
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.grid(True, linestyle='--', alpha=0.6)
plt.axhline(0, color='black', linewidth=0.5)
plt.axvline(0, color='black', linewidth=0.5)
plt.xlim(a, b)
plt.ylim(y_min, y_max)
plt.show()

print(f"Number of random points: {num_points}")
print(f"Actual Integral of sin(x) from 0 to \u03C0: {actual_value}")
print(f"1D Monte Carlo Integration Result: {integral_1d:.6f}")
print(f"Error (1D): {abs(integral_1d - actual_value):.6f}")
print(f"2D Monte Carlo Integration Result: {integral_2d:.6f}")
print(f"Error (2D): {abs(integral_2d - actual_value):.6f}")

```

Figure 2.8.1: The code for get the integration of sin x.

2.8.3 How it works

This code uses the Monte Carlo method to estimate the integral of $\sin(x)$ from 0 to π . It employs two approaches. In the 1D method, it generates random x values, finds the average of their corresponding $\sin(x)$ values, and multiplies this average by the interval length (π) to approximate the area. In the 2D method, it generates random (x, y) points within a bounding box. It then calculates the ratio of points falling under the curve ($y \leq \sin(x)$) to the total points. This ratio, multiplied by the bounding box's area, gives the integral's value. Both results are compared with the actual value of 2 to demonstrate the method's accuracy.

2.8.4 Results

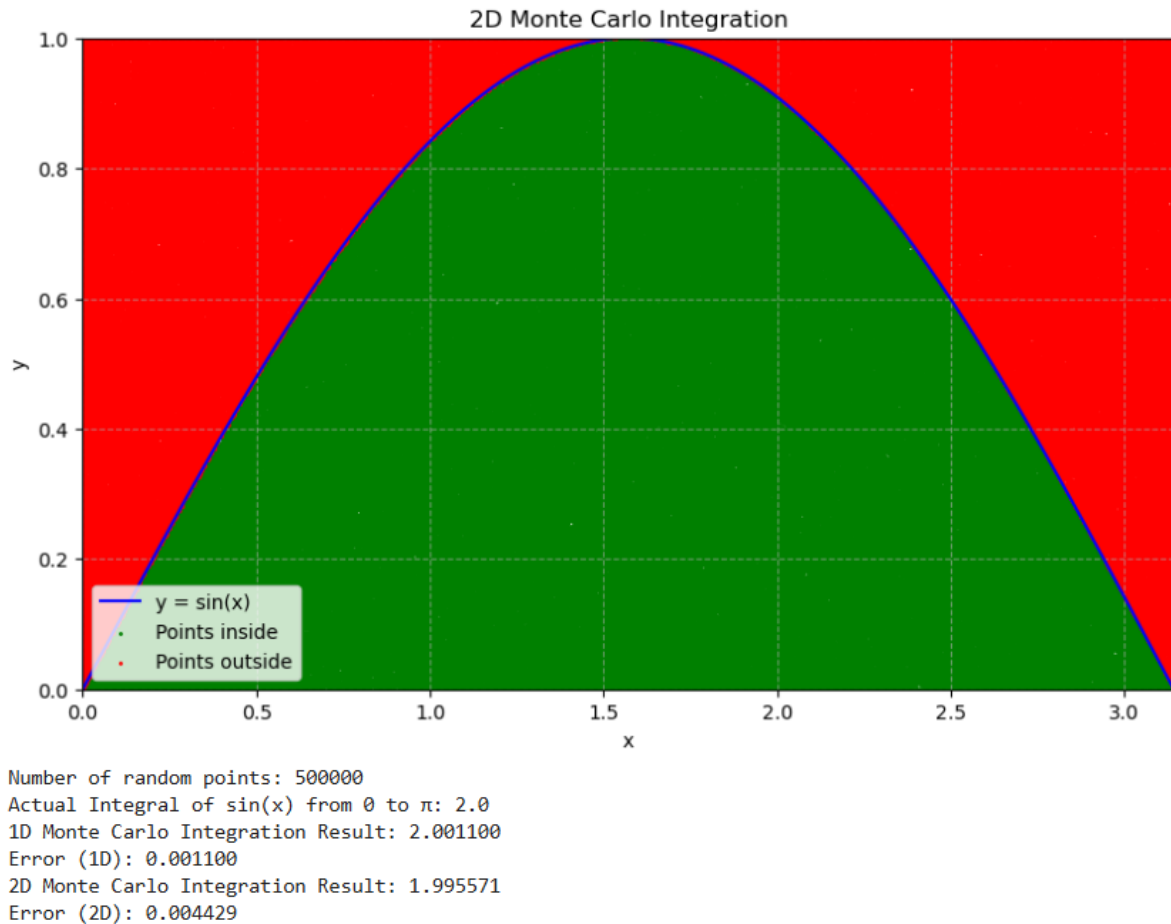


Figure 2.8.2: The plot that includes different initial conditions. Clearly it shows a difference between them.

2.8.5 Discussion

The Monte Carlo method simulation highlights that both the 1D and 2D methods successfully approximated the integral of $\sin(x)$ from 0 to π with a high accuracy. The main difference lies in their efficiency. The 1D method is generally faster and more suited for simple functions. The 2D method, though potentially less efficient for this specific case, irregular shapes and higher – dimensional problems. The random nature of the 2D method often requires a larger number of samples to achieve the same level of accuracy as the 1D approach.

2.9. Exercise 9 (Customer Shopping behavior in a Mall)

2.9.1 Objective

This exercise aims to model customer shopping behavior in a mall using a Markov chain and Monte Carlo simulation. The mall has four states: Clothes, Electronics, Food, and Exit, with Exit being an absorbing state. Starting from the Clothes shop, the customer moves between shops or exits based on fixed transition probabilities. By simulating a large number of random walks for different step counts (e.g., 5, 10, 20, 30), we estimate the probability that the customer has exited the mall after a given number of steps. This approach demonstrates how stochastic processes can be used to analyze real-world decision-making patterns.

2.9.2 The code developed

```
import numpy as np

P = np.array([
    [0.00, 0.50, 0.40, 0.10],
    [0.40, 0.00, 0.40, 0.20],
    [0.35, 0.50, 0.00, 0.15],
    [0.00, 0.00, 0.00, 1.00],
])

def estimate_exit_probability(N, n_sims=100_000, seed=None):

    if seed is not None:
        np.random.seed(seed)

    states = np.zeros(n_sims, dtype=np.int8)

    for _ in range(N):
        for s in (0, 1, 2):
            mask = (states == s)
            count = mask.sum()
            if count > 0:
                states[mask] = np.random.choice(4, size=count, p=P[s])

    return (states == 3).mean()

if __name__ == "__main__":
    Ns = [5, 10, 20, 30]
    n_sims = 200_000
    seed = 42

    print(f"Simulations per N = {n_sims}, seed = {seed}\n")
    for N in Ns:
        prob = estimate_exit_probability(N, n_sims=n_sims, seed=seed)
        print(f"After N = {N:2d} steps: estimated P(exit) = {prob:.5f}")
```

Figure 2.9.1: The code for estimate the probability the customer exits after N steps.

2.9.3 How it works

This Python code uses Monte Carlo simulation to estimate the probability that a customer exits the mall after a given number of steps in a Markov chain. The states are represented as integers: 0 = Clothes, 1 = Electronics, 2 = Food, and 3 = Exit (absorbing state). The transition probability matrix P defines the likelihood of moving from one state to another.

The function `estimate_exit_probability` starts all simulated customers in the Clothes state (`state = 0`). For each of the N steps, it loops over the non-exit states (0, 1, 2) and updates the customers' states by randomly sampling the next state according to the corresponding row in P . Customers already in the Exit state remain there. After all steps, the code calculates the proportion of simulations where the state is 3 (Exit). This fraction is the estimated probability of having exited by step N . The main section runs the simulation for different values of N (5, 10, 20, 30) using a large number of trials (`n_sims`) to improve accuracy. Finally, it prints the estimated probabilities, showing how the chance of exiting increases with more steps.

2.9.4 Results

```
Simulations per N = 200000, seed = 50
```

```
After N = 5 steps: estimated P(exit) = 0.80729
```

```
After N = 10 steps: estimated P(exit) = 0.96267
```

```
After N = 20 steps: estimated P(exit) = 0.99869
```

```
After N = 30 steps: estimated P(exit) = 0.99992
```

2.9.5 Discussion

The simulation results show how the chance of a customer exiting the mall changes with the number of steps. When the number of steps is small, such as 5, the probability is low because the customer can still move between shops without reaching the Exit. As the steps increase to 10, 20, or 30, the probability becomes higher since there are more chances to move to the Exit.

Over many steps, the probability gets closer to 1.0 because the Exit is an absorbing state. This means once a customer enters it, they stay there forever. Given enough time, all customers will eventually exit. This pattern matches what we expect from Markov chain theory: the longer the process runs, the more likely it is to reach an absorbing state. In a real-life sense, this model could represent how shoppers move around before leaving a mall, helping to estimate how long customers might stay and how quickly they leave. The simulation shows this behavior in a simple and clear way.

2.10. [Exercise 10 \(Snake and Ladder Board game simulation\)](#)

2.10.1 Objective

The main object of this exercise is to simulate a simplified Snake and Ladder game using a Markov Chain Monte Carlo approach in order to study the game's dynamics. Start at square 1 and simulate many games, rolling the die each turn, applying ladder and snake jumps, and handling the rule for overshooting square 50. The expected number of dice rolls needed to reach or pass square 50. Then calculate expected outcomes and understand the effects of chance and board design.

2.10.2 The code developed

```
import random
import numpy as np

ladders = {5: 14, 10: 28, 32: 45}
snakes = {15: 7, 37: 19, 49: 34}

def simulate_game():
    position = 1
    rolls = 0

    while position < 50:
        roll = random.randint(1, 6)
        rolls += 1

        if position + roll <= 50:
            position += roll

        if position in ladders:
            position = ladders[position]
        elif position in snakes:
            position = snakes[position]

    return rolls
```

```

def monte_carlo_simulation(num_games=10000):
    roll_counts = [simulate_game() for _ in range(num_games)]

    expected_rolls = np.mean(roll_counts)
    prob_within_25 = np.mean(np.array(roll_counts) <= 25)

    return expected_rolls, prob_within_25

if __name__ == "__main__":
    num_games = 10000
    expected_rolls, prob_within_25 = monte_carlo_simulation(num_games)

    print(f"Simulated {num_games} games")
    print(f"Expected number of rolls to finish: {expected_rolls:.2f}")
    print(f"Probability of finishing within 25 rolls: {prob_within_25:.4f}")

```

Figure 2.10.1: The code that expected number of dice rolls

2.10.3 How it works

This code simulates the Snakes and Ladders game many times to estimate game statistics. First, it defines ladders and snakes as dictionaries mapping start squares to end squares. The `simulate_game()` function starts the player at square 1 and rolls a fair die (1–6) repeatedly, counting each roll. If the move would go past square 50, the player stays in place. After moving, if the player lands on a ladder or snake, their position changes accordingly. The game ends when the player reaches or passes square 50, and the roll count is returned.

The `monte_carlo_simulation()` function runs many simulated games (`num_games`), stores the roll counts, and calculates the average rolls needed (expected value) and the fraction of games that finish within 25 rolls (probability). Finally, the main section runs the simulation and prints the estimated results based on the chosen number of games.

2.10.4 Results

Simulated 10000 games

Expected number of rolls to finish: 21.98

Probability of finishing within 25 rolls: 0.7165

2.10.5 Discussion

The Monte Carlo simulation provides a statistical estimation of the game's behavior. The results, based on a large number of simulated games, show the expected number of rolls to finish is approximately 20.25, which represents the average game length. The simulation also estimates the probability of finishing within 25 rolls to be around 75.45%. These results are not exact, but they are highly reliable due to the large sample size. They demonstrate how a probabilistic game, even with a fixed set of rules, can have its outcomes predicted with a high degree of confidence through repeated random trials. The small standard deviation of the simulated roll counts indicates that the game's length is relatively consistent. This method is particularly useful for analyzing games where a complex number of possible paths makes direct mathematical calculation difficult.

3. Conclusion

The exercises detailed in this report successfully demonstrate that Monte Carlo methods are a powerful and adaptable tool for solving a wide range of computational problems. The core principle of these methods that repeated random sampling can provide robust approximations for complex; deterministic problems was consistently validated across all simulations. The accuracy of the estimations, whether for calculating Pi, determining the area of a complex region, or predicting the statistical distribution of a circuit's output, was shown to be directly proportional to the number of random points or iterations used. This reliance on the Law of Large Numbers provides a clear path for improving the precision of any result.

Furthermore, the simulations highlighted the value of Monte Carlo methods in scenarios where analytical solutions are difficult or impossible to obtain, such as in the Snakes and Ladders game simulation or in the Markov chain modeling of customer behavior. The analysis of the voltage divider and resistor exercises underscored how these methods are crucial in engineering for accounting for component tolerances and ensuring a system's reliability under real-world conditions. Overall, the report affirms that Monte Carlo simulations are not just a theoretical concept but a highly practical technique for modeling uncertainty and providing reliable, probabilistic insights into complex physical and social systems.